URLs to external APIs, all depending on the environment to which you're deploying the application. Cloud native applications favor externalized configuration so that you can replace it without having to rebuild your code. Regarding credentials, it's even more critical not to store them with your application code. There have been countless data breaches because of companies including credentials in a repository that went public by accident. Make sure you won't be one of them.

In Spring, configuration data is abstracted as properties (key/value pairs) defined in different sources, such as property files, JVM system properties, and system environment variables. This chapter covers various aspects of configuring Spring applications that are relevant in a cloud native context. I'll first present the main concepts behind how Spring handles configuration, including properties and profiles, and how you can apply externalized configuration with Spring Boot. Then I'll show you how to set up a configuration server with Spring Cloud Config Server, using a Git repository as the backend for storing your configuration data. Finally, you'll learn how to use the config server to configure a Spring Boot application by relying on Spring Cloud Config Client.

By the end of this chapter, you'll be able to configure your cloud native Spring applications in different ways, depending on your needs and the type of configuration data you have. Table 4.1 summarizes the three main strategies for defining configuration data for cloud native applications covered in this chapter. Chapter 14 will further extend the subjects covered here, including secrets management and how to use ConfigMaps and Secrets in Kubernetes.

**NOTE** The source code for the examples in this chapter is available in the `Chapter04/04-begin` and `Chapter04/04-end` folders, which contain the initial and final states of the project (https://github.com/ThomasVitale/cloud-native-spring-in-action).